

Lecture: Jump Search Algorithm — From Zero to Advanced

1. Introduction

Jump Search is an efficient searching algorithm designed for **sorted arrays**.

Instead of checking elements one by one (like Linear Search), it **jumps ahead by fixed steps** and then performs a **linear scan** within a block when the possible range is identified.

Definition:

Jump Search reduces the number of comparisons by skipping $\sqrt{n}$ elements at a time until the target is possibly found.

Real-life analogy:

Imagine searching for a word in a dictionary by flipping pages in **fixed jumps** (say 20 pages at a time). Once you overshoot the word, you flip back a few pages linearly until you find it.

2. Key Characteristics

- **Type:** Block-based Search (hybrid of Linear Search and Divide & Conquer)
- **Input Requirement:** Data must be **sorted**
- **Direction:** Jumps in fixed intervals (commonly $\sqrt{n}$)
- **Time Complexity:** $O(\sqrt{n})$
- **Space Complexity:** $O(1)$

3. Step-by-Step Working

1. Let n = length of array, choose block size = $\sqrt{n}$.
2. Start at index 0, jump ahead by block size until the target $\leq$ current element or end is reached.
3. Perform a **linear search** backward in the identified block.
4. If found $\rightarrow$ return index. Otherwise $\rightarrow$ return -1.

4. Pseudocode

```
import math

def jump_search(arr, target):
    n = len(arr)
    step = int(math.sqrt(n)) # optimal step size
    prev = 0

    # Jump until we reach a block where target could be
    while prev < n and arr[min(step, n) - 1] < target:
        prev = step
        step += int(math.sqrt(n))
        if prev >= n:
            return -1 # target not found

    # Linear search within the block
    while prev < min(step, n):
        if arr[prev] == target:
            return prev
        prev += 1

    return -1 # not found
```

5. Complexity Analysis

- **Best Case:** $O(1)$ → target found at the first jump
- **Worst Case:** $O(\sqrt{n})$ → jump across blocks + linear scan

- **Average Case:** $O(\sqrt{n})$
- **Space Complexity:** $O(1)$

6. Handling Edge Cases

1. **Empty list** → return -1
2. **Single element list** → check directly
3. **Multiple occurrences** → modify linear search to find first/last occurrence
4. **Unsorted input** → invalid results

7. Variations of Jump Search

- **Optimal block size:** usually $\sqrt{n}$, but can be tuned based on dataset.
- **Exponential Search + Jump Search:** Jump size increases exponentially until overshoot, then linear search.
- **Block Search in Linked Lists:** A modified form for block-linked data structures.

8. When to Use Jump Search

Best suited for:

- Large sorted datasets where random access is possible
- Systems where linear access is cheap but binary search overhead is costly

Avoid if:

- Data is unsorted
- Binary Search is more efficient (typical in memory-based searches)

9. Advanced Usage

- Searching in **disk-based systems** (where sequential access is faster than random access).
- Used as a **pre-step for Exponential Search**.
- Useful in memory-limited embedded systems where recursion (Binary Search) is not preferred.

10. Real-World Applications

- **Databases:** Index searching in sorted blocks

- **Search engines:** Optimizing access when binary search is costly
- **Library systems:** Searching sorted catalogs with limited memory
- **Disk storage:** Jumping through fixed-size blocks before scanning linearly

11. Summary Table

Feature	Jump Search
Works on sorted?	Yes
Works on unsorted?	No
Time Complexity	$O(\sqrt{n})$
Space Complexity	$O(1)$
Best Case	$O(1)$
Worst Case	$O(\sqrt{n})$

12. Practice Tasks

1. Implement Jump Search iteratively in Python.
2. Modify Jump Search to return the **first and last occurrence** of an element.
3. Compare Jump Search, Linear Search, and Binary Search on datasets of size 10^5 .
4. Implement Jump Search for strings in a sorted dictionary.
5. Experiment with different block sizes and analyze performance.

13. Closing Thought

Jump Search provides a balance between **Linear Search ($O(n)$)** and **Binary Search ($O(\log n)$)**.

Though Binary Search is generally faster, Jump Search is valuable in **disk-based systems** and environments where block-level access is more efficient.

Mastering it helps in understanding **search optimization** when both time and memory constraints matter.